

Jogo

Campo minado

Next Slide

Apresentação do projeto

Nosso projeto é um jogo de Campo Minado desenvolvido em JavaScript. O objetivo é revelar todas as células que não possuem bombas sem clicar em uma bomba. O jogo possui três dificuldades: Fácil, Médio e Difícil, além de cinco modos diferentes.

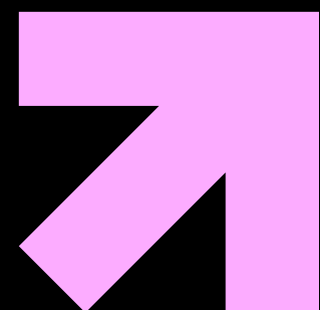
As dificuldades são:

- Fácil: 6×6 com 5 bombas.
- Médio: 8×8 com 10 bombas.
- Difícil: 10×10 com 20 bombas.

OS MODOS SÃO:

- **NORMAL;**
- **SEGURO NA PRIMEIRA JOGADA**
- **ÁREA SEGURA;**
- **CONTRA O TEMPO;**
- **HARDCORE.**

ESTRUTURAS CONDICIONAIS



No nosso projeto utilizamos estruturas condicionais para fazer o programa tomar decisões. As principais são if, else if e else

```
if (opcao === 1) {  
  LINHAS = 6;  
  COLUNAS = 6;  
  BOMBAS = 5;  
} else if (opcao === 3) {  
  LINHAS = 10;  
  COLUNAS = 10;  
  BOMBAS = 20;  
} else {  
  LINHAS = 8;  
  COLUNAS = 8;  
  BOMBAS = 10;  
}
```

Nesse trecho, o if verifica se o jogador escolheu a opção 1. O else if verifica se escolheu a opção 3. Se nenhuma dessas condições for verdadeira, o else define a dificuldade como Médio



ESTRUTURA DE DADOS-MATRIZ

```
var tabuleiro = [ ];  
var linha = [ ];
```

Também utilizamos uma estrutura de dados em forma de matriz para representar o tabuleiro. A matriz possui linhas e colunas e cada posição representa uma célula do Campo Minado

```
for (var i = 0; i < LINHAS; i++) {  
    var linha = [ ];  
  
    for (var j = 0; j < COLUNAS; j++) {  
        linha.push(valor);  
    }  
  
    tabuleiro.push(linha);  
}
```

O primeiro for cria as linhas e o segundo for cria as colunas. Assim conseguimos montar o tabuleiro de acordo com a dificuldade escolhida.



FUNCIONAMENTO DO JOGO

O funcionamento começa pelo menu principal, onde o jogador pode escolher entre iniciar uma partida ou visualizar o histórico. Ao iniciar, ele coloca seu nome, escolhe a dificuldade e o modo de jogo. Depois escolhe uma linha e uma coluna do tabuleiro.

```
while (!jogoAcabou) {  
    exibirTabuleiro(tabuleiroVisivel);  
  
    var linha = readline.questionInt('Linha: ');  
    var coluna = readline.questionInt('Coluna: ');  
}
```

O while mantém o jogo funcionando enquanto a partida não tiver terminado. A cada rodada, o jogador escolhe uma linha e uma coluna.

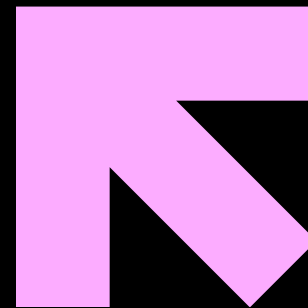
ESTRUTURAS DE REPETIÇÃO

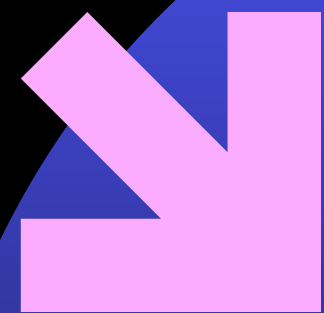
```
while (bombasColocadas < BOMBAS) {
```

Esse while é responsável por repetir o processo de colocar bombas. Ele continua executando enquanto a quantidade de bombas colocadas for menor que a quantidade definida.

```
for (var i = -1; i <= 1; i++) {  
  for (var j = -1; j <= 1; j++) {
```

Esses for são usados para verificar as células que ficam ao redor da posição escolhida e descobrir quantas bombas existem próximas.





Organização do código

FUNCTION ESCOLHERDIFICULDADE()

FUNCTION ESCOLHERMODOJOGO()

FUNCTION CRIARTABULEIRO()

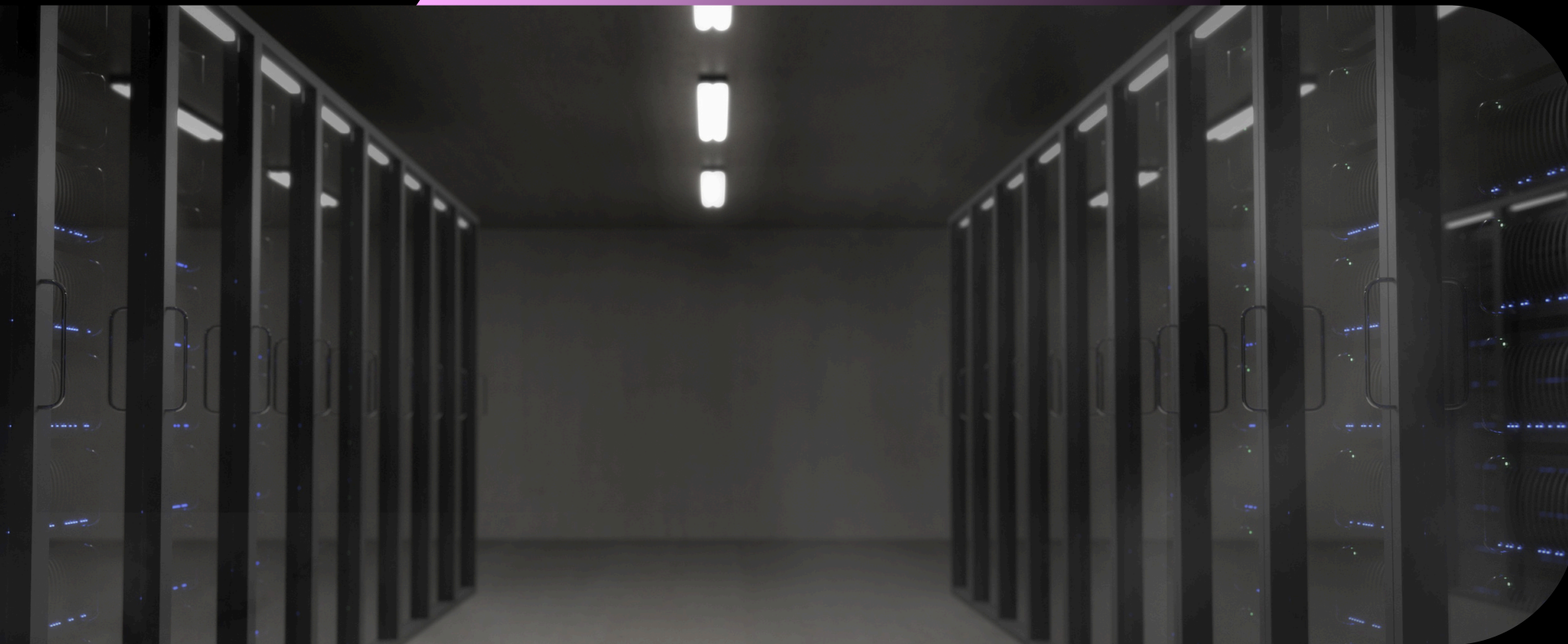
FUNCTION POSICIONARBOMBAS()

FUNCTION REVELARCELULA()

FUNCTION VERIFICARVITORIA()

FUNCTION JOGAR()

Para organizar melhor o projeto, dividimos o código em várias funções. Cada função possui uma responsabilidade. Por exemplo, `posicionarBombas()` coloca as bombas, `revelarCelula()` revela uma posição e `verificarVitoria()` verifica se o jogador ganhou.



HISTÓRICO DAS PARTIDAS

```
var ARQUIVO_RANKING = './ranking.json';
```

```
fs.writeFileSync(  
  ARQUIVO_RANKING,  
  JSON.stringify(ranking, null, 2),  
  'utf-8'  
);
```

Também criamos um sistema de histórico. Utilizamos o módulo fs para salvar os resultados em um arquivo chamado ranking.json. Dessa forma, conseguimos guardar informações como nome do jogador, vitória ou derrota, tempo, dificuldade e modo.

OBRIGADO
PELA
ATENÇÃO